

Azure DevOps Interview Guide

Essential Questions and Answers for
Your Next DevOps Role



By Prasanjt Sahoo

1. How do you design a highly available and scalable architecture in Azure for a web application?

To design a **highly available and scalable** Azure architecture:

- **Use Azure Front Door or Application Gateway:** Acts as the entry point, provides load balancing and web application firewall (WAF).
- **Deploy in multiple regions** using **Availability Zones** to ensure regional failure tolerance.
- **Use Azure App Services or AKS** (for containers) with **autoscaling** rules based on CPU/memory/requests.
- **Store data in Azure SQL with Geo-Replication** or Cosmos DB for multi-region reads/writes.
- **Use Azure Traffic Manager or Front Door** for failover and routing traffic intelligently.

Key Concepts: Load Balancing, Auto-scaling, Geo-redundancy, Multi-region deployments.

2. What's the difference between Azure VM Availability Sets and Availability Zones?

Feature	Availability Sets	Availability Zones
Scope	Within a single data center	Across physically separate zones
Fault Domains	Multiple fault & update domains	Each zone has independent power, network
Uptime SLA	99.95%	99.99%
Use Case	Legacy VM workloads	Business-critical apps needing high resilience

Use **Availability Zones** for higher fault tolerance across datacenters.

3. How would you troubleshoot a failed Azure Resource deployment?

1. **Check Activity Log:** In Azure Portal → Monitor → Activity Log for errors.
2. **Deployment Logs:** Go to the resource group → Deployments → View failed operations.
3. **Template/Parameter Issues:** Validate ARM/Bicep/Terraform syntax and values.
4. **Permissions:** Ensure the user/service principal has the right RBAC roles.
5. **Resource Quotas:** Check if you're hitting region-based limits (e.g., VM cores).
6. **Region Availability:** Make sure that SKUs are available in selected region.

Best Practice: Use az deployment commands with --debug flag in CLI for detailed logs.

4. What is Azure Key Vault and how do you use it in CI/CD pipelines securely?

Azure Key Vault is a secure service to store:

- Secrets (API keys, passwords),
- Certificates, and
- Encryption keys.

Using in CI/CD:

- Grant pipeline identity (managed identity/service principal) **Get/Read** access.
- Use Key Vault secrets in Azure DevOps via **Azure Key Vault task** or **variable groups**.
- In GitHub Actions, access secrets via azure/login + azure/keyvault actions securely.

Never store secrets directly in YAML or environment variables.

5. How do you secure Azure Storage Account access from public exposure?

1. **Disable public access** at the storage account level.
2. **Use Private Endpoints** to restrict access to a VNet.
3. **Enable firewalls** to allow only specific IPs or VNets.
4. **Use SAS Tokens** or **Azure AD-based access** for secure file access.
5. **Enable soft delete & versioning** to recover from accidental deletes.

Always use HTTPS and enable logging for access patterns.

6. What's the difference between ARM, Bicep, and Terraform in Azure? Which one do you prefer and why?

Tool	Description	Pros	Cons
ARM	JSON-based native Azure template	Native support, full Azure feature coverage	Hard to read, verbose
Bicep	DSL (simplified) for ARM	Easier syntax, better tooling, native	Still Azure-specific

Terraform	Multi-cloud IaC tool	Cloud-agnostic, reusable modules, mature ecosystem	Requires provider plugin
------------------	----------------------	--	--------------------------

I prefer:

- **Bicep** for native Azure workloads—clean syntax and built-in support.
- **Terraform** when multi-cloud or advanced modularization is required.

7. Can you write or describe a Bicep file to deploy an Azure Function with an App Service Plan?

```
param location string = resourceGroup().location
param functionAppName string
param storageAccountName string
```

```
resource storage 'Microsoft.Storage/storageAccounts@2022-09-01' = {
  name: storageAccountName
  location: location
  sku: { name: 'Standard_LRS' }
  kind: 'StorageV2'
  properties: {}
}
```

```
resource appPlan 'Microsoft.Web/serverfarms@2022-09-01' = {
  name: '${functionAppName}-plan'
  location: location
  sku: {
    name: 'Y1'
    tier: 'Dynamic'
  }
  properties: {}
}
```

```
resource functionApp 'Microsoft.Web/sites@2022-09-01' = {
  name: functionAppName
  location: location
  kind: 'functionapp'
  properties: {
    serverFarmId: appPlan.id
    siteConfig: {
      appSettings: [
        { name: 'AzureWebJobsStorage', value:
storage.properties.primaryEndpoints.blob }
        { name: 'FUNCTIONS_EXTENSION_VERSION', value: '~4' }
        { name: 'FUNCTIONS_WORKER_RUNTIME', value: 'dotnet' }
```

```
}  
}  
}  
}
```

Use parameter files for different environments.

8. How do you handle parameterization and environment-specific deployments in Terraform or Bicep?

- **Terraform:**
 - Use *.tfvars files per environment (e.g., dev.tfvars, prod.tfvars).
 - Use var.<name> with default values or override via CLI:
terraform apply -var-file="dev.tfvars"
 - Leverage workspaces for environment isolation.
- **Bicep:**
 - Use *.bicepparam files or parameter files in JSON.
 - Deploy using:
az deployment group create --template-file main.bicep --parameters @dev.parameters.json

Best Practice: Keep secrets and env values out of templates and use Key Vault + param files.

9. Walk me through a CI/CD pipeline you've implemented using Azure DevOps or GitHub Actions.

CI/CD Example – Azure DevOps for deploying a .NET app to App Service:

1. **CI Pipeline:**
 - Triggered on code push.
 - Steps:
 - Checkout repo
 - Restore packages
 - Build
 - Run unit tests

- Publish artifacts

2. CD Pipeline:

- Triggered post successful CI.
- Uses Azure Key Vault for secrets.
- Deploys via AzureWebApp task to App Service.
- Includes slot swapping for zero-downtime deploy.

Tools: Azure DevOps Pipelines, Key Vault, App Service, Azure CLI task.

10. How do you manage secrets securely in CI/CD pipelines?

- **Use Azure Key Vault:**
 - Grant pipeline identity access.
 - Retrieve secrets at runtime.
- **In Azure DevOps:**
 - Use variable groups linked with Key Vault.
 - Mask secrets in logs (isSecret: true).
- **In GitHub Actions:**
 - Use **GitHub Secrets** (encrypted at rest).
 - Avoid echoing secrets in logs.
 - Use azure/login + keyvault actions to fetch secrets.

Golden Rule: **Never store secrets in repo** or hardcoded in pipeline YAMLs.

11. What's the difference between Build Pipeline and Release Pipeline in Azure DevOps?

Feature	Build Pipeline	Release Pipeline
Purpose	Compile code, run tests, and create artifacts	Deploy artifacts to environments like dev/prod
Trigger	Automatically on code push	Manual or post-build trigger
Output	Build artifacts (binaries, packages)	Deployed application or infrastructure
Example Tools	MSBuild, dotnet build, npm	ARM/Bicep/Terraform, Azure CLI, Azure App Service

Build = **make code ready**, Release = **get it running** in the environment.

12. How do you implement Infrastructure-as-Code deployment in your pipelines?

1. **Store IaC code** (Bicep, Terraform) in your Git repository.
2. **Build pipeline** validates syntax (e.g., terraform validate, bicep build).
3. **Release pipeline** or CD stage:
 - Authenticate with Azure using **Service Principal** or **Managed Identity**.
 - Run terraform plan or az deployment for preview.
 - Apply using terraform apply or az deployment create.

Bonus: Use approvals, versioning, and secret integration for production deployments.

13. How would you implement blue-green or canary deployments in Azure App Services or AKS?

- **In App Services:**
 - Use **deployment slots** (e.g., “staging” and “production”).
 - Deploy to the staging slot → validate → swap slots for zero downtime.
 - For **canary**, use traffic splitting (e.g., 90% to prod, 10% to staging).
- **In AKS:**
 - **Blue-Green:** Deploy new version to a parallel deployment, then switch service selector.
 - **Canary:** Use **Istio** or **NGINX Ingress Controller** to route partial traffic to new pods.

Best Practice: Automate validations and rollbacks in pipeline after deployment.

14. What are the steps to deploy a containerized application on Azure Kubernetes Service (AKS)?

1. **Build Image:** Use Docker to build your app image.
2. **Push to Registry:** Push to Azure Container Registry (ACR).
3. **Create AKS Cluster** (if not already done).

4. Create Kubernetes Manifests:

- Deployment YAML
- Service YAML (LoadBalancer/ClusterIP)

5. Deploy using kubectl:

- Authenticate with `az aks get-credentials`
- Run `kubectl apply -f deployment.yaml`

6. Monitor: Use kubectl get pods and Azure Monitor for health.

15. How do you manage secrets in AKS?

- **Kubernetes Secrets:**
 - Store base64-encoded secrets, mount as env vars or volumes.
 - Not encrypted by default unless encryption-at-rest is enabled.
- **Azure Key Vault + CSI Driver:**
 - Mount secrets directly into pods using Key Vault via CSI (Container Storage Interface) driver.
 - Secure, centralized, and supports versioning.
- **Use Managed Identity** to grant pod access to Key Vault without storing credentials.

Avoid storing secrets in config maps or images.

16. What is a Helm chart? Have you created or customized one?

Helm is a package manager for Kubernetes—like apt for Ubuntu or npm for Node.js.

- A **Helm Chart** is a collection of YAML templates for deploying Kubernetes resources (Deployments, Services, Ingress, etc.).
- It uses **values.yaml** for environment-specific values.

Yes, I've created and customized charts:

- Created charts for internal microservices.
- Used `helm install` and `helm upgrade`.
- Customized templates with conditionals and loops for different environments.

Helm helps manage complexity and version control for K8s deployments.

17. How do you perform zero-downtime deployments on AKS?

1. **Rolling Update Strategy:**

- Defined in the deployment YAML.
- Kubernetes gradually replaces old pods with new ones.

2. **strategy:**

3. **type: RollingUpdate**

4. **rollingUpdate:**

5. **maxSurge: 1**

6. **maxUnavailable: 0**

7.

8. **Health Probes:**

- Configure readinessProbe and livenessProbe to prevent traffic to unhealthy pods.

9. **Avoid Breaking Changes:**

- Maintain backward compatibility during rollout.

Use kubectl rollout status to monitor progress and rollback if needed.

18. How do you implement role-based access control (RBAC) in Azure?

- **RBAC** lets you assign roles to users, groups, or service principals with specific permissions.
- Example: Assign "Reader" role to someone who needs read-only access.
- Use **Azure AD** identities and assign roles at different scopes:
 - Subscription
 - Resource Group
 - Resource

Use least privilege: Don't assign Owner or Contributor unless truly necessary.

19. Explain the principle of least privilege in the context of Azure and DevOps tools.

The **Principle of Least Privilege (PoLP)** means **giving only the access necessary to perform a task—and nothing more.**

In Azure:

- Don't give full subscription access if resource group access is enough.
- Use **custom roles** if built-in roles are too permissive.

In DevOps tools:

- Limit pipeline identities to deploy only to required services.
- Avoid storing secrets in pipelines—use Key Vault or secure stores.

Benefit: Minimizes blast radius if credentials are compromised.

20. What is Managed Identity, and how does it differ from Service Principal?

Feature	Managed Identity	Service Principal
Identity Type	Managed by Azure	Manually created
Secret Management	No secrets/passwords to manage	Requires manual secret rotation
Use Case	Best for Azure services like VMs, Functions	Used for automation and app-to-Azure access
Lifecycle	Tied to resource lifecycle	Needs manual cleanup

Best Practice: Use **Managed Identity** whenever possible—it's more secure and low-maintenance.

21. How do you monitor application and infrastructure performance in Azure?

To monitor application and infrastructure performance in Azure, I typically use **Azure Monitor**. It collects and analyzes telemetry data from both the app and the underlying infrastructure.

For applications, I use **Application Insights**, it helps track live performance, exceptions, request rates, and response times.

For infrastructure, I monitor metrics like CPU, memory, disk I/O, and network traffic on Azure VMs or services.

I also set up **log analytics workspaces** to query logs and create custom dashboards. This gives a holistic view and helps detect performance bottlenecks or failures early.

22. What's your approach to setting up alerts and automated remediation in Azure?

First, I identify critical metrics and logs that indicate system health, such as CPU usage, error rates, or failed requests.

Then, I create **Azure Monitor alerts** on those metrics, setting thresholds for warning and critical levels.

For example, if CPU usage stays above 80% for 5 minutes, an alert triggers.

For automated remediation, I use **Azure Automation Runbooks** or **Logic Apps** that get triggered by alerts to perform corrective actions like scaling out VMs, restarting services, or clearing caches.

This reduces manual intervention and downtime. I also integrate alerts with communication tools like Teams or email for immediate visibility.

23. A pipeline deployment failed. How would you investigate and fix it quickly?

First, I check the **pipeline logs** to identify exactly where the failure happened—build, test, or deployment stage.

The logs usually give error messages or stack traces that hint at the root cause. If it's a build failure, I look into code errors, dependency issues, or environment mismatches.

For deployment failures, I verify resource permissions, configurations, and infrastructure readiness.

Next, I replicate the failure locally or in a staging environment to reproduce and debug.

If the problem relates to credentials or access, I check service connections or secret management.

Finally, I fix the issue whether it's code, configuration, or pipeline definition—and rerun the pipeline to validate the fix.

24. Explain how Azure Virtual Network integrates with on-prem via VPN or ExpressRoute.

Azure Virtual Network (VNet) can connect securely to on-premises networks using either **VPN Gateway** or **ExpressRoute**.

- **VPN Gateway** uses an encrypted IPsec tunnel over the public internet to connect the Azure VNet to on-premises network devices. It's relatively quick and cost-effective but depends on internet reliability.
- **ExpressRoute** is a private, dedicated connection between on-premises infrastructure and Azure datacenters. It offers higher bandwidth, lower latency, and better security because traffic doesn't traverse the public internet.

Both methods let on-premises servers communicate with Azure resources as if they're on the same network, supporting hybrid cloud setups.

25. You get a request to create 100 identical VMs for load testing. How would you automate it?

I would automate this using **Infrastructure-as-Code (IaC)** tools like **Terraform** or **Azure Bicep**. I'd write a template defining the VM size, image, network settings, and storage.

Then, I'd use a loop or count parameter to specify the number of VMs 100 in this case, so all get deployed uniformly.

Alternatively, I could use **Azure CLI scripts** with a for loop or **Azure PowerShell** scripts to create VMs programmatically.

For better management, these VMs would be part of a **Virtual Machine Scale Set (VMSS)**, which automatically handles scaling and load balancing.

Using VMSS simplifies managing large numbers of identical VMs.